

Redpwn 2020 Crypto - Alien Transmissions v2

deuterium — 2020-06-27

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/redpwn/crypto/alien-transmissions-v2/2020-06-27-redpwn-2020-alien-transmissions-v2.html>

Abstract

CTF writeup: Even aliens have word-spacing habits. Frequency analysis recovers a combined XOR key in redpwnCTF 2020's Alien Transmissions v2; Z3 splits it in two.

Alien-transmissions-v2

Description

The aliens are at it again! We've discovered that their communications are in base 512 and have transcribed them in base 10. However, it seems like they used XOR encryption twice with two different keys! We do have some information:

- * This alien language consists of words delimited by the character represented as 481
- * The two keys appear to be of length 21 and 19
- * The value of each character in these keys does not exceed 255

Find these two keys for me; concatenate their ASCII encodings and wrap it in the flag format.

Files

- [encrypted.txt](#) (HUGE number of characters)

Observations

- Hint 1, the delimited character in plaintext is represented as 481. How does it help? Because in any language delimiter of words (space in our case) is the most frequent character (although we can't be sure about the aliens, what if they have really long words??), we can assume that the plaintext character 481 is the most frequent.
- Hint 2 states that it is XORed twice, once with the key of length 21 and again by length of key 19. If one ponders more, the two keys would act as a combined key of length of LCM of the two lengths 19 and 21, which is $21 \times 19 = 399$. And given the huge number of words (1100151), we can recover the key at least pretty accurately.

Combined XOR key recovery

The plaintexts at a difference of keylength are always encrypted by the same key letter, so splitting the ciphertext in keylength number of chunks, we can get a whole chunk, which has been encrypted by the same key byte. And since the character frequency of each chunk

would resemble the character frequency distribution of the whole text owing to the huge size of ciphertext.

Just find the character with maximum frequency in each chunk, that would be 481 XOR key, XOR it with 481, to get the corresponding compound key character

```
with open('encrypted.txt','r') as encrypted:
    data = list(map(int, encrypted.readlines()))

key = bytearray()
for i in range(21*19):
    data_slice = data[i::21*19]
    key_char_val = 481 ^ max(data_slice, key=data_slice.count)
    key.append(key_char_val)

print(key)
```

It takes a couple of seconds to find the compound key

```
key = bytearray(b'7G\x1a\x00\x00\x17X];9\x00Gj\x007Y\x013\x12\x00\x00-\x06\x14Vo\x1a\x0c\nSn\x06]Ej7@\x04U7\x06AP\x17[;+Y\x06\x00\x12YC\x00++\x00\x073S[PB]CjnA7G7W\x04-A\x1c17\x01_\x05X]\x16l\x16\x00\x00\x00\x00\x02j9E\x1a\x06+j[W\\x08\x0c1C\x06xA7E]l+\x0e\x02-\x00G<XZ\x089Y\x06-G0\x04j+\x1c[l9\x04AVD1\x0ck]S7G\x1a\x02\x12j+\x1c[ljURB[\x10\x00Y\x013\x12\x00\x02GLS]l+]\x00<V_\x16jEj7@\x04W]l\x06[\x14jjG\x0b\x031\x02nC\x00++\x00\x05Y9\x1c[Al\x12\x06<D\x06W\x00W\x04-A\x1cn]k\x18\x0e[lG\x00D\x051\x107\x02j9E\x1a\x04A\x00\x1c\\_9]\x00\x11\x03IQ\x00E]l+\x0e\x00Gj\x007[kYU\x0b\x03\x1cWx\x04j+\x1c[nSn\x06]G\x00]\x07\x0fV\x06W-\x02\x12j+\x1cY\x06\x00\x12YAjAl\x0b\x04\x02\x027\x02GLS]nA7G7UnG\x06\x17o\x06P3W]l\x06[\x16\x00\x00\x00\x00\x00\x00S\x02\x11\x05\x1a;7\x05Y9\x1c[C\x06xA7G7\x06l\x05\x01\x1cCQ+n]k\x18\x0eY\x06-G0\x06\x00A[Po\x08U-\x04A\x00\x1c\\]S')
```

Now, we have the XOR of key1 and key2, we need to recover key1 and key2 from it. We have all possible bytes, of key1 XORed with key2 and vice-versa.

First thought, Z3 go brrrrrr....

Although Z3 produces a feasible solution(model), a quick [stackOverflow](#) post gave me the implementation of finding n models.

```
from z3 import *
def get_models(F, M):
    """
    Returns a list of M models (if possible) from the list of constraints F
    """
    result = []
    s = Solver()
    s.add(F)
    while len(result) < M and s.check() == sat:
        m = s.model()
        result.append(m)
        # Create a new constraint the blocks the current model
        block = []
```

```

    for d in m:
        # d is a declaration
        if d.arity() > 0:
            raise Z3Exception("uninterpreted functions are not supported")
        # create a constant from declaration
        c = d()
        if is_array(c) or c.sort().kind() == Z3_UNINTERPRETED_SORT:
            raise Z3Exception("arrays and uninterpreted sorts are not supported")
        block.append(c != m[d])
    s.add(Or(block))
return result

```

All we need to do is to form constraints and checking if the generated model produces ASCII printable strings.

```

key1 = [z3.BitVec("k1{}".format(i),8) for i in range(21)]
key2 = [z3.BitVec("k2{}".format(i),8) for i in range(19)]
F = []
for i in range(19*21):
    F.append(key1[i%21]^key2[i%19]==key[i])

# Valid flag characters
VALID_CHARS = string.printable[0:62]+"_.,'?!@$<>*:~]*\\"
for model in get_models(F,256):
    # 256 models to get all possible relative strings
    KEY1 = "".join( chr(model[key1[i]].as_long()) for i in range(21))
    KEY2 = "".join( chr(model[key2[i]].as_long()) for i in range(19))
    flag = KEY1+KEY2
    if all(i in VALID_CHARS for i in flag):
        print(flag)

```

Turns out there was only one model with all characters printable and which is our flag

flag-h3r3'5'th3'flr5t'h4lf'th3'53c0nd'15'th15"

| Interactive controls are omitted from this PDF. See the canonical HTML version.